

Before beginning work on this assignment, carefully read the Assignment Submission Specifications posted on Canvas.

Introduction

As noted in chapter 8, the number of possible keys for the basic transposition cipher is equal to half the length of the message. This means that, unless the message is extremely long, it is still possible to use the brute force technique to decipher a message enciphered with the transposition cipher, even without the key. This exercise will guide you to implementing a stronger cipher, much more resistant to brute force attacks.

The transposition cipher enciphers a message by writing the message down in rows, and then reading of the columns, from left to right. For example, to encipher the message “CIPHERS ARE FUN” with the key 5, we create the following table. We then read the columns left to right to get the ciphertext “CREIS P FHAUERN”.

```
C I P H E
R S _ A R
E _ F U N
```

Instead of always reading the columns left to right, suppose we read the columns of in the order 4, 5, 3, 1, 2. We would get the ciphertext “HAUERNP FCREIS ” (note the trailing space character). On the other hand, if we choose the order 2, 4, 1, 5, 3, we get the ciphertext “IS HAUCREERNP F”. In fact, the number of distinct orders in which we can read these five columns is 5! (120). Even if a hacker knows we used five columns in our table, they still wouldn’t know which of the 120 possible orders will allow them to break the code.

Better still, the number of possible orders increases very quickly with the number of columns, since the factorial function grows extremely fast. If we used ten columns instead of five, the number of possible orders is over three million. If we used fifteen, the number of orders is over one trillion. The brute force technique will fail as long as the number of columns is not too small.

Note that the cipher’s key is no longer a single number, but a particular permutation of the numbers from 1 up to a particular number. For example, “2, 4, 1, 5, 3” is a possible key to this cipher. This modification – changing the key from a number to a permutation, and reading the columns in that order – results in the *columnar transposition cipher*.

For your implementation, use TRANSPOSITIONENCRYPT.PY and TRANSPOSITIONDECRYPT.PY from the textbook source files as references.

You will produce a total of five files for this assignment:

- **a2p1.py, a2p2.py, a2p3.py, a2p4.py**: python solutions to problems 1, 2, 3, and 4 respectively
- a README file (**README.txt**)

Submit your files in a zip file named 331-as-2.zip.

Problem 1

While the columnar transposition cipher is our eventual goal, we will begin with a simpler version: the regular columnar cipher, discussed above. Here, the columns are *not* permuted before being read off into ciphertext. For this first problem, your task is to modify the provided skeleton file, “a2p1.py” to implement this type of cipher. You will implement two functions named “encipherMessage” and “decipherMessage” each taking two arguments:

- key: an integer between 2 and the length of the message to determine how many columns are to be used
- message: a string

For Problems 1–3, the input message may contain uppercase and lowercase letters, spaces, digits, punctuation, and other characters. Preserve all characters and their case exactly as provided. Do not convert the message to uppercase or lowercase.

Your encipherMessage function should return the message enciphered with the transposition cipher using the number of columns specified by the key. Your decipher function should undo the cipher and return the original plaintext. The following are examples of functions calls and their expected outputs:

```
>>> encipherMessage(5, "CIPHERS ARE FUN")
"CREIS P FHAUERN"
>>> decipherMessage(5, "CREIS P FHAUERN")
"CIPHERS ARE FUN"
```

Problem 2

For this problem, you will be implementing a columnar transposition cipher. Modify the provided skeleton file, “a2p2.py” to implement a function named “encipherMessage” that takes two arguments:

- key: a list of integers which determine the order in which the columns are to be rearranged
- message: a plaintext string to be encrypted

Every integer in the key is unique and is between 1 and the number of columns. For example, encipherMessage([2, 4, 1, 5, 3], “CIPHERS ARE FUN”) should return “IS HAUCREERNP F”.

If the length of the given message is not a multiple of the number of columns, your program should use the shaded box technique described in the textbook. The following shows an example of a function call and its expected output:

```
>>> encipherMessage([2, 4, 1, 5, 3], "CIPHERS ARE FUN")
'IS HAUCREERNP F'
```

Problem 3

Now that you can encipher using the columnar transposition cipher, the next task is to be able to automatically decipher messages. Modify the provided skeleton file, “a2p3.py” so that it implements a function named “decipherMessage” that takes two arguments:

- key: the list of integers that was used to encrypt the original plaintext
- message: a cipher string to be deciphered

To test your code, try deciphering messages that were enciphered using the program from Problem 2, for example:

```
>>> decipherMessage([2, 4, 1, 5, 3], "IS HAUCREERNP F")
'CIPHERS ARE FUN'
```

Problem 4

Your final task is to crack a columnar transposition cipher given several ciphertext words and a known key length. Modify the provided skeleton file, “a2p4.py”, to implement a function named “crackSharedKey”. Your implementation will likely call the code implemented in the previous problems.

The function “crackSharedKey” takes two arguments:

- **keylength**: an integer denoting the length of the key (i.e. the number of columns)
- **cipherWords**: a list of ciphertext words, each enciphered using the same key

Recall that the columnar transposition cipher preserves the set of letters in the plaintext. Your program should use the provided **dictionary.txt** file to identify candidate plaintexts for each ciphertext, and determine which keys could have produced them.

For example, given the ciphertext “AET” and a key length of 3, the plaintext may have been “ATE”, in which case the key was [1, 3, 2], “EAT”, in which case the key was [2, 1, 3], or “TEA” using key [3, 2, 1]. If multiple ciphertexts are available, this information can be combined. For instance, the ciphertext “WSA” could have been enciphered using the keys [3, 1, 2], or [1, 3, 2]. The only key consistent with both ciphertexts is therefore [1, 3, 2]. Note that we need a dictionary which includes the relevant plaintexts to be able to deduce this.

The function should return a list of all keys that could have been used to encipher *every* ciphertext in **cipherWords**. If no such key exists, the function should return an empty list. When multiple keys are returned, their order does not matter. The following examples demonstrate correct behavior:

```
>>> crackSharedKey(3, ["AET"])
[[1, 3, 2], [2, 1, 3], [3, 2, 1]]
>>> crackSharedKey(3, ["AET", "WSA"])
[[1, 3, 2]]
>>> crackSharedKey(3, ["AET", "WSA", "OSM"])
[]
```